

MERN Real-Time Chat App

Backend System Design + AI Coding Prompt

Purpose: A simple one-to-one chatting web application where authenticated users can search for other users by username, start conversations, exchange real-time messages, and see online/offline status.

1. Technology Stack

Layer	Technology	Purpose
Runtime	Node.js	Backend runtime
API	Express.js	REST API and middleware
Database	MongoDB + Mongoose	Users, chats, messages
Authentication	JWT + bcrypt	Login/session authentication and password hashing
Real-time	Socket.IO	Instant messaging, typing and presence
Validation	Zod or Joi	Validate request payloads
Security	Helmet, CORS, rate limiting	Basic API hardening

2. High-Level Architecture

React Frontend → Express REST API → Controllers/Services → MongoDB
React Frontend ↔ Socket.IO Server ↔ Connected Users
JWT Authentication protects private API routes and Socket.IO connections.

3. Backend Folder Structure

```
backend/  
  ■■■ src/  
    ■ ■■■ config/db.js  
    ■ ■■■ controllers/  
      ■ ■ ■■■ authController.js  
      ■ ■ ■■■ userController.js  
      ■ ■ ■■■ chatController.js  
      ■ ■ ■■■ messageController.js  
    ■ ■■■ middleware/  
      ■ ■ ■■■ authMiddleware.js  
      ■ ■ ■■■ errorMiddleware.js  
      ■ ■ ■■■ rateLimitMiddleware.js  
    ■ ■■■ models/  
      ■ ■ ■■■ User.js  
      ■ ■ ■■■ Chat.js  
      ■ ■ ■■■ Message.js  
    ■ ■■■ routes/  
      ■ ■ ■■■ authRoutes.js  
      ■ ■ ■■■ userRoutes.js  
      ■ ■ ■■■ chatRoutes.js  
      ■ ■ ■■■ messageRoutes.js  
    ■ ■■■ services/  
      ■ ■ ■■■ chatService.js  
      ■ ■ ■■■ messageService.js  
    ■ ■■■ socket/socket.js  
    ■ ■■■ utils/generateToken.js  
    ■ ■■■ server.js  
  ■■■ .env  
  ■■■ package.json  
  ■■■ .gitignore
```

4. MongoDB Data Model

User

`_id`, `username` (unique/indexed), `email` (unique), `passwordHash`, `avatar`, `isOnline`, `lastSeen`, `createdAt`, `updatedAt`

Chat

`_id`, `participants`: [`UserId`, `UserId`], `lastMessage`, `createdAt`, `updatedAt`. For the MVP, a chat is strictly one-to-one.

Message

`_id`, `chat`, `sender`, `text`, `read`, `createdAt`, `updatedAt`. Add `media`/`reply`/`edit`/`delete` fields later only when required.

5. REST API Design

Method	Endpoint	Purpose
POST	/api/auth/register	Create account
POST	/api/auth/login	Authenticate user
GET	/api/auth/me	Get current user
GET	/api/users/search?q=	Search users by username
GET	/api/users/:id	Get public user profile
POST	/api/chats	Create or return existing one-to-one chat
GET	/api/chats	Get current user's chats
GET	/api/chats/:id	Get chat details
DELETE	/api/chats/:id	Delete/archive chat if supported
GET	/api/messages/:chatId	Get paginated chat messages
POST	/api/messages	Optional REST fallback for creating a message
PATCH	/api/messages/:id/read	Mark message as read
DELETE	/api/messages/:id	Delete own message if enabled

6. Socket.IO Event Design

```
Client → Server:
join_chat { chatId }
leave_chat { chatId }
send_message { chatId, text }
typing { chatId }
stop_typing { chatId }
message_read { messageId, chatId }
```

```
Server → Client:
message_received { message }
message_sent { message }
user_online { userId }
user_offline { userId, lastSeen }
typing_started { userId, chatId }
typing_stopped { userId, chatId }
message_read { messageId, userId }
error { message }
```

7. Authentication & Authorization Flow

1. Register: validate input → check username/email → bcrypt hash password → save user.
2. Login: validate credentials → compare password hash → issue JWT.
3. REST: auth middleware verifies Bearer JWT and sets req.user.
4. Socket.IO: authenticate the socket with JWT during connection.
5. Authorization: verify the authenticated user belongs to a chat before reading/sending messages.
6. Never return passwordHash in API responses.

8. Core Message Flow

```
User A searches for User B
→ GET /api/users/search
→ User A selects User B
→ POST /api/chats
→ Server returns existing chat or creates one
```

```
→ Client loads GET /api/messages/:chatId
→ Both users join the Socket.IO chat room
→ User A emits send_message
→ Server validates authorization
→ Server saves message to MongoDB
→ Server emits message_received to the room
→ User B sees the message immediately.
```

9. Security Requirements

- Hash passwords with bcrypt; never store plaintext passwords.
- Keep JWT_SECRET only on the server.
- Validate and sanitize request bodies.
- Use Helmet and strict CORS configuration.
- Add rate limiting to authentication and user-search endpoints.
- Check chat membership before returning messages or accepting socket messages.
- Prevent users from impersonating another sender; sender comes from authenticated identity.
- Use pagination for message history.
- Do not expose database errors or secrets to clients.
- Use HTTPS/WSS in production.
- Add MongoDB indexes for username/email and chat/message lookup fields.

10. Environment Variables

```
PORT=5000
MONGO_URI=mongodb+srv://...
JWT_SECRET=
JWT_EXPIRES_IN=7d
CLIENT_URL=http://localhost:5173
```

11. MVP Development Order

- Phase 1 — Project setup + MongoDB connection
- Phase 2 — User registration/login + JWT
- Phase 3 — User search by username
- Phase 4 — Chat creation and chat list
- Phase 5 — Message persistence and pagination
- Phase 6 — Socket.IO real-time messaging
- Phase 7 — Online/offline presence
- Phase 8 — Typing indicators + read receipts
- Phase 9 — Security hardening + error handling
- Phase 10 — Deployment and production environment variables

12. AI Coding / Backend Generation Prompt

Build a production-ready MVP backend for a simple MERN real-time one-to-one chat application.

TECH STACK

- Node.js
- Express.js
- MongoDB with Mongoose
- JWT authentication
- bcrypt password hashing
- Socket.IO for real-time communication
- Zod or Joi for validation
- Helmet, CORS and express-rate-limit for security

CORE FEATURES

1. User registration and login.
2. Secure password hashing with bcrypt.
3. JWT authentication.
4. Protected REST endpoints.
5. Search users by username.
6. Start a one-to-one conversation with another user.
7. Prevent duplicate conversations between the same two users.
8. Retrieve the authenticated user's chat list.
9. Retrieve paginated message history.
10. Send messages in real time using Socket.IO.
11. Persist every message in MongoDB.
12. Online/offline presence and lastSeen.
13. Typing indicators.
14. Message read status.
15. Centralized error handling.
16. Secure environment-variable configuration.

PROJECT STRUCTURE

Create a clean layered structure:

```
src/config
src/controllers
src/middleware
src/models
src/routes
src/services
src/socket
src/utils
server.js
```

DATABASE MODELS

User:

- username: unique, required, indexed
- email: unique, required
- passwordHash: required
- avatar: optional
- isOnline: boolean
- lastSeen: date
- timestamps

Chat:

- participants: exactly two User references
- lastMessage: Message reference
- timestamps
- enforce logic that a pair of users has only one conversation

Message:

- chat: Chat reference
- sender: User reference
- text: required string
- read: boolean
- timestamps

REST API

```
POST /api/auth/register
POST /api/auth/login
GET /api/auth/me
GET /api/users/search?q=
GET /api/users/:id
POST /api/chats
GET /api/chats
GET /api/chats/:id
GET /api/messages/:chatId
PATCH /api/messages/:id/read
DELETE /api/messages/:id
```

SOCKET.IO

Implement:
join_chat

leave_chat
send_message
typing
stop_typing
message_read

Emit:
message_received
user_online
user_offline
typing_started
typing_stopped
message_read
error

REAL-TIME MESSAGE FLOW

When send_message is received:

1. Authenticate the socket.
2. Validate the payload.
3. Confirm the sender belongs to the chat.
4. Save the message to MongoDB.
5. Update Chat.lastMessage and updatedAt.
6. Emit the saved message to the appropriate chat room.
7. Return a clear acknowledgement/error.

AUTHORIZATION

A user must only be able to:

- access their own profile/session data
- access chats they participate in
- read messages belonging to their chats
- send messages as themselves
- mark messages in their accessible chats as read

SECURITY

- Never store plaintext passwords.
- Never return passwordHash.
- Keep JWT_SECRET server-side.
- Validate all request/socket payloads.
- Configure CORS using CLIENT_URL.
- Use Helmet.
- Rate-limit authentication and search endpoints.
- Sanitize/validate user input.
- Do not trust senderId supplied by the client.
- Use authenticated user identity as the sender.
- Return safe error messages.
- Add indexes for frequent queries.
- Use pagination for messages.

DEVELOPER EXPERIENCE

Provide:

- package.json
- .env.example
- complete source files
- MongoDB connection
- Express app setup
- Socket.IO setup
- middleware
- models
- controllers
- routes
- services
- error handling
- README with installation and API usage
- example Socket.IO client events

Do not implement unnecessary features such as group chats, payments, stories, video calls, or complex media uploads. Keep the architecture simple, modular, secure, and easy to extend later.

13. Expected Backend Result

The finished backend should provide a clean API and Socket.IO layer that the React frontend can consume. The MVP should support the complete journey: register → login → search username → open chat → load history → send/receive real-time messages → show online status → mark messages read.